

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Branch: CSM

Section: B

Task No.: 2

Student Name: K. Saranya

Roll No.: A24126552090

1. TITLE

Design and Develop a Pure Semantic HTML Static Webpage Using Structural Tags, Tables for Layout, and Native Form Elements Without CSS.

2. AIM

To design and develop a static webpage using pure semantic HTML5 elements — without any CSS — that demonstrates a header with navigation, a hero section, a two-column article/sidebar layout built with a table, native form controls, and a footer, relying entirely on the browser's default rendering.

3. OBJECTIVE

The objectives of this experiment are:

- To understand how a webpage can be structured using only semantic HTML tags.
- To use header, nav, section, main, article, aside, and footer to organise a document.
- To build a navigation bar using anchor links that jump to in-page sections.
- To use a table for the layout of a two-column content area without CSS.
- To create a form with fieldset, legend, labels, and various input types.
- To apply HTML5 form validation attributes such as required and placeholder.
- To observe how a browser renders structural HTML using only its own default styles.
- To verify that the page stays readable and functional with no stylesheet at all.

4. THEORY

Semantic HTML means using HTML elements according to the meaning of the content they hold rather than just for their default appearance. Tags such as header, nav, main, article, aside, and footer describe the role each block plays in the page, which helps browsers, search engines, and screen readers understand its structure.

When no CSS is linked, the browser falls back to its own built-in user-agent stylesheet. Headings, paragraphs, tables, and form controls are all rendered using these defaults, which is why a page like this still looks organised even without a single styling rule.

- header, nav, footer mark the top banner, the navigation links, and the closing section of the page.
- section, main, article, aside divide the body into a hero area, the main content region, individual posts, and a side panel.

- A table used purely for layout arranges the header row and the two-column body area using rows and cells — a technique common before CSS layout existed.
- code marks inline text as source code, and the browser renders it in a monospace font by default.
- fieldset and legend group related form controls together and give the group a visible caption.
- required and placeholder are HTML5 attributes that add built-in validation and hint text without any JavaScript.

The basic flow of the page is:

Browser requests the HTML file → Browser applies its default stylesheet → Semantic structure is rendered as a readable page

5. ALGORITHM / PROCEDURE

1. Create a new HTML file named static_webpage.html.
2. Add the DOCTYPE declaration, the html tag, and a head section with a title.
3. Build a header containing the site name and a table-based navigation bar with anchor links.
4. Add a horizontal rule to visually separate the header from the page body.
5. Create a hero section with a centred heading, an introductory paragraph, and a call-to-action link.
6. Create a structural table with two columns to hold the main content and the sidebar.
7. In the left column, add a main element containing two article blocks with headings and paragraphs.
8. Inside the second article, build a form using fieldset, legend, labels, and input, select, and textarea controls.
9. In the right column, add an aside element listing benefits and required components using unordered and ordered lists.
10. Add a footer with a centred copyright notice.
11. Save the file and open it in a web browser without linking any CSS file.
12. Verify that the layout, navigation, and form controls render correctly using only browser defaults.

6. PROGRAM

```
static_webpage.html
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <title>Pure Semantic HTML Static Webpage</title>
6  </head>
7  <body>
8
9      <!-- Header Section -->
10     <header>
11         <!-- A borderless table is used here only to place the site name and the nav
12         bar side by side without any CSS -->
13         <table width="100%" border="0" cellspacing="0" cellpadding="10">
14             <tr>
15                 <td>
16                     <h1>StaticWeb</h1>
17                 </td>
18                 <td align="right">
19                     <nav>
20                         <!-- Each link jumps to an id further down the same page -->
```

```

20         <a href="#home">Home</a> &nbsp;&nbsp;&nbsp;|&nbsp;&nbsp;&nbsp;
21         <a href="#articles">Articles</a> &nbsp;&nbsp;&nbsp;|&nbsp;&nbsp;&nbsp;
22         <a href="#about">About</a> &nbsp;&nbsp;&nbsp;|&nbsp;&nbsp;&nbsp;
23         <a href="#contact">Contact Us</a>
24     </nav>
25 </td>
26 </tr>
27 </table>
28 </header>
29
30 <hr>
31
32 <!-- Hero / Showcase Section -->
33 <section id="home">
34     <center>
35         <!-- <center> is an old but still browser-supported way to centre content
without CSS -->
36         <h2>Semantic Structures Without CSS</h2>
37         <p>This layout uses raw HTML components to build an organized document
hierarchy that browsers can cleanly parse out-of-the-box.</p>
38         <p><a href="#articles"><b>Read Our Articles Below &darr;</b></a></p>
39     </center>
40 </section>
41
42 <hr>
43
44 <!-- Main Content Layout using a Structural Table -->
45 <table width="100%" border="0" cellspacing="0" cellpadding="20">
46     <tr valign="top">
47
48         <!-- Left Side: Main Content Column -->
49         <td width="70%">
50             <main id="articles">
51
52                 <article>
53                     <h2>1. Core Structural Tags</h2>
54                     <p>Published: July 7, 2026</p>
55                     <p>When you omit cascading style sheets entirely, the
semantic markup handles all of the structural heavy lifting. Elements like
<code>&lt;main&gt;</code>, <code>&lt;article&gt;</code>, and
<code>&lt;section&gt;</code> tell screen readers and web scrapers exactly how
information flows.</p>
56                     <p>Using raw browser defaults ensures excellent loading
performance, perfect accessibility compliance, and absolute structural clarity.</p>
57                 </article>
58
59                 <br><hr><br>
60
61                 <article>
62                     <h2>2. Native Interaction Elements</h2>
63                     <p>Published: July 5, 2026</p>
64                     <p>Browsers come equipped with excellent interactive elements
by default. We can capture text inputs, provide multiple-choice configurations, and
submit structured information to backend parsers using pure markup.</p>
65
66                     <h3>Interactive Demonstration Forms</h3>
67                     <form id="contact" action="#" method="post">
68                         <fieldset>

```

```

69         <!-- <fieldset> and <legend> group the form controls
and give the group a visible caption -->
70         <legend><b>User Inquiry Form</b></legend>
71         <p>
72             <label for="name">Full Name: </label>
73             <input type="text" id="name" name="username"
required placeholder="John Doe">
74         </p>
75         <p>
76             <label for="email">Email Address: </label>
77             <input type="email" id="email" name="useremail"
required>
78         </p>
79         <p>
80             <label for="topic">Inquiry Topic: </label>
81             <select id="topic" name="usertopic">
82                 <option value="general">General
Feedback</option>
83                 <option value="technical">Technical
Architecture</option>
84                 <option value="academics">Academic
Curriculum</option>
85             </select>
86         </p>
87         <p>
88             <label for="message">Message Details:</label><br>
89             <textarea id="message" name="usermessage"
rows="5" cols="40" placeholder="Type your notes here..."></textarea>
90         </p>
91         <p>
92             <input type="submit" value="Submit Form Data">
93             <input type="reset" value="Clear Entries">
94         </p>
95         </fieldset>
96     </form>
97 </article>
98
99 </main>
100 </td>
101
102 <!-- Right Side: Sidebar Column -->
103 <td width="30%" bgcolor="#f4f4f4">
104     <aside id="about">
105         <h3>Document References</h3>
106         <p>Static pages map data out directly without real-time database
queries or processor overhead.</p>
107
108         <h4>Core Benefits:</h4>
109         <ul>
110             <li>Instant browser painting</li>
111             <li>Low memory footprints</li>
112             <li>High data compatibility</li>
113         </ul>
114
115         <h4>Required Components:</h4>
116         <ol>
117             <li>Document Type Definition</li>
118             <li>Semantic Wrapper Blocks</li>
119             <li>Data Input Control Interfaces</li>

```

```

120         </ol>
121     </aside>
122 </td>
123
124 </tr>
125 </table>
126
127 <hr>
128
129 <!-- Footer Section -->
130 <footer>
131     <center>
132         <p>&copy; 2026 Pure HTML Architecture. Assembled completely without style
configurations.</p>
133     </center>
134 </footer>
135
136 </body>
137 </html>

```

7. CODE EXPLANATION

Tag / Attribute	Description
<header>, <nav>, <footer>	Mark the top banner, the navigation links, and the closing section of the page.
<table> (layout use)	Arranges the nav bar and the two-column body using rows and cells, without any CSS.
<section>, <main>, <article>, <aside>	Divide the body into a hero area, the primary content region, individual posts, and a side panel.
	Creates an in-page link that jumps to the element with the matching id.
<code>	Displays inline text in a monospace font, typically used for code snippets.
<fieldset>, <legend>	Group related form controls and give the group a visible caption.
<label for="id">	Associates descriptive text with a form control that shares the same id.
required	HTML5 built-in validation that blocks submission if the field is left empty.
placeholder	Shows light hint text inside a field before the user types anything.
<select>, <option>	Create a dropdown list of choices for the Inquiry Topic field.
<textarea rows cols>	Provides a resizable multi-line text box for longer messages.
, ,	List the sidebar's Core Benefits and Required Components.

8. TEST CASES AND EXECUTION DATA

The page was opened in a browser and each structural and interactive element was tested individually; the results were recorded below.

Test Case	Action	Expected Result	Actual Result	Status
TC-01	Open static_webpage.html in a browser	Page loads showing header, nav, hero, articles, sidebar, and footer	Page displayed correctly	Pass
TC-02	Click 'Articles' in the navigation bar	Browser jumps down to the Articles section	Jumped to correct section	Pass
TC-03	Leave the Full Name field empty and click Submit	Browser blocks submission and prompts for the field	Required-field prompt shown	Pass
TC-04	Enter an invalid email format	Browser shows an email-format validation error	Validation error displayed	Pass
TC-05	Open the Inquiry Topic dropdown	All three options are listed and selectable	Dropdown worked correctly	Pass
TC-06	Type multiple lines in Message Details	Textarea accepts and displays multi-line text	Multi-line text accepted	Pass
TC-07	Click 'Clear Entries'	All form fields reset to their default blank state	Fields cleared correctly	Pass

Additional Validation Test

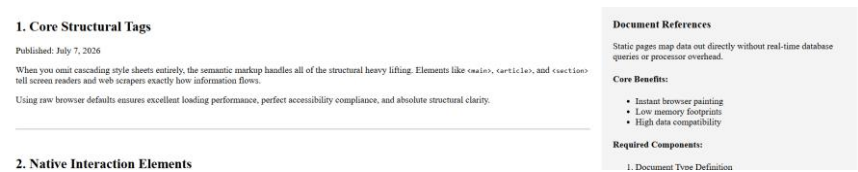
Test Case	Input / Action	Expected Result	Status
TC-08	Resize the browser window	Table-based layout remains readable without a stylesheet	Pass
TC-09	View the page with a screen reader	Semantic tags are announced in a logical reading order	Pass

9. OUTPUT

Output 1: Header, Navigation Bar, and Hero Section



Output 2: Article 1 – Core Structural Tags (with sidebar)



Output 3: Article 2 – Interactive User Inquiry Form

Published: July 5, 2026

Browsers come equipped with excellent interactive elements by default. We can capture text inputs, provide multiple-choice configurations, and submit structured information to backend parsers using pure markup.

Interactive Demonstration Forms

User Inquiry Form

Full Name:

Email Address:

Inquiry Topic: General Feedback ▼

Message Details:

2. Semantic Wrapper Blocks

3. Data Input Control Interfaces

Submit Form Data

Clear Entries

© 2026 Pure HTML Architecture. Assembled completely without style configurations.

Output 4: Form Buttons, Sidebar Tail, and Footer

10. RESULT

Thus, a pure semantic HTML static webpage was successfully designed and developed without using any CSS. The header, navigation, hero section, table-based two-column layout, articles, interactive form, sidebar, and footer were implemented using semantic tags and were verified to render and function correctly using only the browser’s default styling.